
Informe de Funcionalidades – HealUp

Versión 2.0 (Rediseño desde Cero)

Autor: Benjamín A. Cantero

Fecha: 11 Febrero 2026

1.- OBJETIVO DEL SISTEMA

HealUp es una aplicación móvil de seguimiento y acompañamiento en procesos de rehabilitación física, diseñada para:

- Mejorar la adherencia al tratamiento
 - Visualizar progreso real y medible
 - Motivar mediante logros y recompensas en la app
 - Registrar historial clínico funcional y la rehabilitación del usuario
-

2.- MÓDULOS PRINCIPALES DEL SISTEMA

La aplicación debe dividirse en los siguientes módulos funcionales:

1. Gestión de Usuarios
 2. Gestión de Lesiones
 3. Gestión de Rutinas y Ejercicios
 4. Registro de Progreso Diario
 5. Sistema de Métricas y Estadísticas
 6. Sistema de Logros y premiación
 7. Sincronización y Persistencia
 8. Configuración y Perfil
-

3.- FUNCIONALIDADES DETALLADAS

3.1 Gestión de Usuarios

Funciones necesarias:

- Registro de usuario
- Inicio de sesión
- Cierre de sesión
- Recuperación de contraseña
- Edición de perfil básico
- Eliminación de cuenta
- Persistencia de sesión

Datos mínimos requeridos:

- Email
 - Contraseña
 - Fecha de registro
 - Nombre (opcional)
 - Edad (opcional)
 - Sexo (opcional)
 - Nivel de actividad (opcional)
-

3.2 Gestión de Lesiones

El usuario puede tener múltiples lesiones activas o históricas.

Funciones necesarias:

- Crear nueva lesión
- Editar lesión
- Marcar lesión como recuperada
- Eliminar lesión
- Asociar rutina a lesión

Datos mínimos:

- Nombre de la lesión
 - Fecha de inicio
 - Nivel de dolor inicial (escala 1–10)
 - Zona corporal
 - Estado (activa / recuperada)
-

3.3 Gestión de Rutinas y Ejercicios

Sistema que organiza el tratamiento.

Funciones necesarias:

- Asignar rutina a lesión
- Listar ejercicios diarios
- Modificar rutina
- Cambiar intensidad
- Activar/desactivar ejercicios

Datos requeridos por ejercicio:

- Nombre
 - Descripción
 - Repeticiones
 - Series
 - Duración
 - Nivel de dificultad
 - Recomendaciones
-

3.4 Registro de Progreso Diario

Este es el corazón del sistema.

Debe permitir registrar:

- Ejercicio realizado (sí/no)
- Fecha
- Nivel de dolor antes y después

- Duración real
- Notas del usuario

Debe guardar historial completo.

Nunca sobrescribir datos antiguos.

3.5 Sistema de Métricas y Estadísticas

Funciones necesarias:

- Cálculo de racha de días consecutivos
- Total de ejercicios completados
- Promedio de adherencia semanal
- Evolución del dolor
- Progreso por lesión
- Cumplimiento mensual

Debe generar:

- Métricas diarias
 - Métricas semanales
 - Métricas acumuladas
-

3.6 Sistema de Logros (Gamificación)

Basado en métricas reales.

Funciones:

- Desbloqueo automático de logros
- Seguimiento de progreso hacia meta
- Visualización de logros bloqueados
- Sistema de niveles (opcional)

Ejemplos:

- 7 días consecutivos
- 50 ejercicios completados
- 30 días sin abandonar rutina
- 100% cumplimiento semanal

Los logros deben calcularse dinámicamente.

3.7 Sincronización y Persistencia

Funciones críticas:

- Guardado local inmediato
- Sincronización automática cuando hay red
- Manejo de conflictos de datos
- Respaldo en servidor
- Restauración en nuevo dispositivo

Debe ser offline-first.

3.8 Configuración

Funciones necesarias:

- Notificaciones recordatorias
 - Hora preferida de ejercicio
 - Idioma
 - Modo oscuro
 - Exportación de datos (PDF/CSV futuro)
-

4.- REQUISITOS NO FUNCIONALES

Muy importantes.

Seguridad:

- Hash de contraseñas
- Autenticación JWT
- Encriptación local básica
- Validación de datos

Rendimiento:

- Carga rápida
- Persistencia eficiente

- Bajo consumo de batería

Escalabilidad:

- Soporte multiusuario
- Backend desacoplado
- API versionada

Mantenibilidad:

- Clean Architecture
 - Separación de responsabilidades
 - Testing obligatorio
-

5.- FLUJO FUNCIONAL GENERAL DEL USUARIO

1. Usuario se registra
 2. Crea lesión
 3. Se asigna rutina
 4. Realiza ejercicios diariamente
 5. Registra progreso
 6. Visualiza estadísticas
 7. Desbloquea logros
 8. Recupera lesión
 9. Historial queda guardado
-

6.- DEFINICIÓN DE MVP REAL

Para que HealUp funcione correctamente como MVP mínimo necesita:

Registro/Login

Crear lesión

Asignar rutina

Registrar ejercicio diario

Visualizar progreso semanal

Cálculo de racha

Persistencia local

Sincronización básica

Todo lo demás es expansión.

Conclusión Arquitectónica

HealUp no es una app de checklist.

Es un sistema de:

- Gestión de rehabilitación
- Seguimiento clínico personal
- Motivación conductual

Si lo diseñas bien desde el inicio, puede evolucionar a:

- Plataforma para fisioterapeutas
- Sistema B2B
- Startup HealthTech

1.- Visión Arquitectónica General

HealUp será desarrollado bajo una arquitectura moderna, desacoplada y escalable compuesta por:

- Cliente móvil Android
- Backend REST API
- Base de datos relacional
- Infraestructura containerizada

Se adopta una arquitectura **cliente-servidor con enfoque offline-first**.

2.- Arquitectura General del Sistema

```
graph TD; A[Android App (Kotlin)] --> B[REST API (FastAPI)]; B --> C[PostgreSQL];
```

La comunicación será mediante HTTP/HTTPS utilizando JSON.

3.- Arquitectura del Cliente (Android)

3.1 Lenguaje

- **Kotlin**

Estándar oficial de Android, seguro, moderno y mantenible.

3.2 Framework de Interfaz

- **Jetpack Compose**
- **Material 3**

Arquitectura declarativa para UI moderna, reactiva y escalable.

3.3 Patrón Arquitectónico

- **Clean Architecture**

- **MVVM**
- Gestión de estado con **StateFlow**

Separación en capas:

presentation (UI)

domain (lógica de negocio)

data (fuentes de datos)

Principios aplicados:

- **SOLID**
 - Separación de responsabilidades
 - Inversión de dependencias
-

3.4 Inyección de Dependencias

- **Hilt (Dagger)**

Permite desacoplamiento, testabilidad y escalabilidad.

3.5 Persistencia Local

- **Room**

Utilizado como:

- Base de datos local
 - Cache
 - Soporte offline-first
 - Control de sincronización
-

3.6 Comunicación con Backend

- **Retrofit**
 - **OkHttp**
 - Serialización con Kotlin Serialization o Moshi
-

3.7 Procesos en Segundo Plano

- **WorkManager**

Para sincronización automática de datos.

3.8 Testing

- JUnit
- Mockk
- Compose UI Testing

Testing desde la capa de dominio.

4.- Arquitectura del Backend

4.1 Lenguaje

- **Python 3.11+**
-

4.2 Framework Principal

- **FastAPI**

Razones:

- Alto rendimiento
 - Documentación automática (Swagger)
 - Validación de datos con Pydantic
 - Desarrollo rápido
 - Arquitectura limpia
-

4.3 Arquitectura Interna Backend

Se adoptará una arquitectura modular inspirada en Clean Architecture:

api/
schemas/
services/
repositories/
models/

core/

Separación de responsabilidades:

- api → Endpoints REST
 - schemas → Validación de datos
 - services → Lógica de negocio
 - repositories → Acceso a datos
 - models → Modelos ORM
 - core → Configuración, seguridad, utilidades
-

4.4 Base de Datos

- **PostgreSQL**

Justificación:

- Relacional
 - Integridad referencial
 - Ideal para métricas y relaciones complejas
 - Escalable
-

4.5 ORM

- **SQLAlchemy**

Para mapeo objeto-relacional desacoplado.

4.6 Migraciones

- **Alembic**

Control de versiones del esquema de base de datos.

4.7 Autenticación y Seguridad

- JWT (Access + Refresh Tokens)
- Hashing de contraseñas con bcrypt
- Validación estricta de datos

- Versionado de API (/api/v1)
-

5.-Infraestructura

5.1 Contenerización

- **Docker**
- **Docker Compose**

Servicios iniciales:

- Backend
 - PostgreSQL
-

5.2 Entorno de Desarrollo

- Ubuntu (Linux)
 - Git
 - GitHub
-

6.- Principios Arquitectónicos Adoptados

- Clean Architecture
 - Separación de capas
 - Inversión de dependencias
 - Offline-first
 - API versionada
 - Desacoplamiento frontend/backend
 - Escalabilidad horizontal futura
-

7.- Estrategia de Escalabilidad

El sistema estará preparado para:

- Migración a infraestructura cloud
- Balanceo de carga

- Separación futura en microservicios (si es necesario)
- Integración con servicios externos

Inicialmente se mantendrá como **monolito modular escalable**.

8.- Decisiones Técnicas Clave

- No se utilizará Firebase como backend principal.
 - No se utilizarán microservicios en fase inicial.
 - No se emplearán bases de datos NoSQL.
 - Se priorizará arquitectura mantenible sobre rapidez de desarrollo.
-

9.- Conclusión

HealUp v2 se desarrollará bajo un stack moderno, profesional y escalable compuesto por:



Kotlin + Compose + Clean Architecture



FastAPI + PostgreSQL



Docker + Ubuntu

Esta combinación permite:

- Desarrollo ágil
 - Arquitectura mantenible
 - Escalabilidad futura
 - Profesionalización del proyecto
-